Class 07: Machine Learning 1

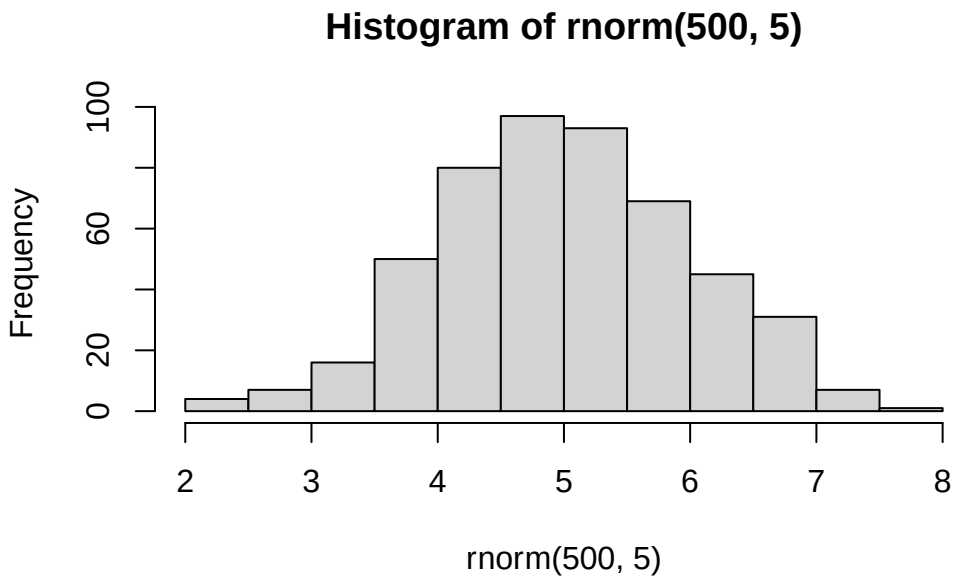
Alejandro Ostos PID: A17978684

Today we will explore some fundamental machine learning methods including clustering and dimensionality reduction.

K-means clustering

To see how this works let's first makeup some data to cluster where we know what the answer should be. We can use the `rnorm()` function to help here:

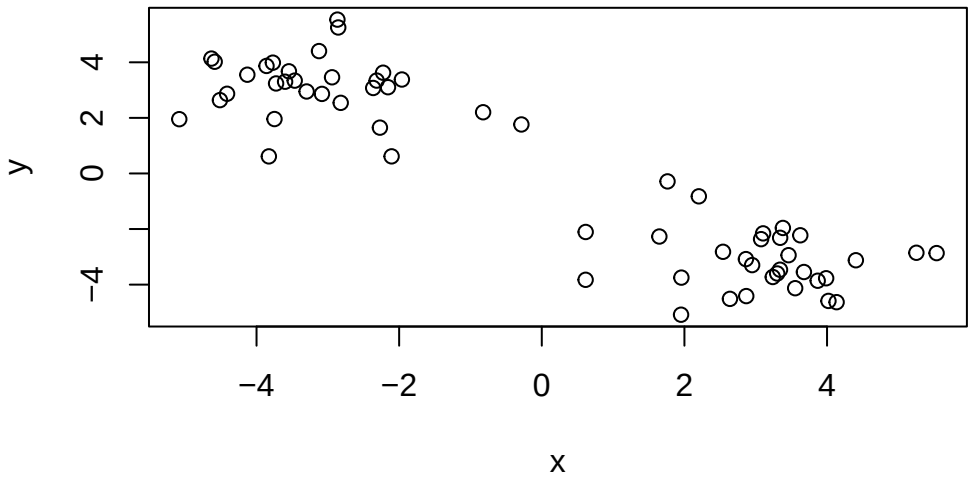
```
hist(rnorm(500, 5))
```



```r
x <- c(rnorm(30, -3), rnorm(30, 3))
y <- rev(x)
```

```r
x <- cbind(x, y)
```

```r
plot(x)
```



The function for K-means clustering in "base" R is `kmeans()`

```r
k <- kmeans(x, centers = 2)
k
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1 -3.146334  3.097066
2  3.097066 -3.146334

Clustering vector:
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
```

```
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

Within cluster sum of squares by cluster:
[1] 72.39828 72.39828
 (between_SS / total_SS =  89.0 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

To get the results of the returned list object we can use the dollar `$` syntax.

Q. How many dots are in each cluster?

```
k$size
```

```
[1] 30 30
```

Q. What 'component' of your result object details - cluster assignment/membership?
- cluster center?

```
k$cluster
```

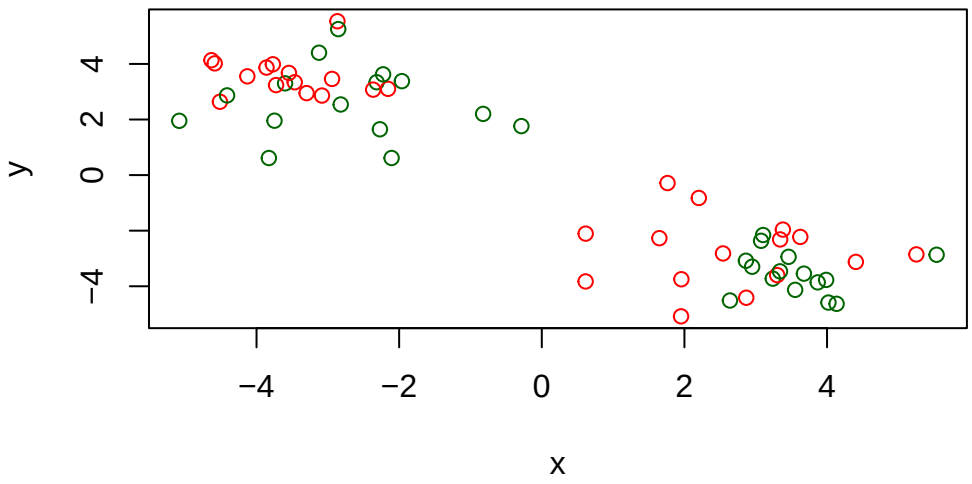
```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
k$centers
```

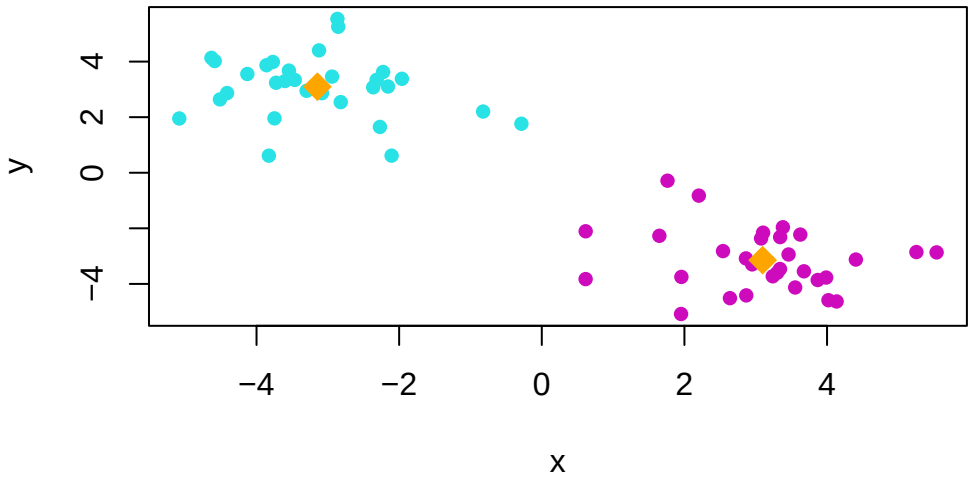
```
          x         y
1 -3.146334  3.097066
2  3.097066 -3.146334
```

Q. Make a clustering results figure of the data colored by cluster membership and show membership and show cluster centers.

```
plot(x, col = c("red", "darkgreen"))
```




```
plot(x, col = 4 + k$cluster, pch = 16)
points(k$centers, col = "orange", pch = 18, cex = 2)
```



K-means clustering is very popular as it is very fast and relatively straight forward: it takes numeric data as input and returns the cluster membership vector etc.

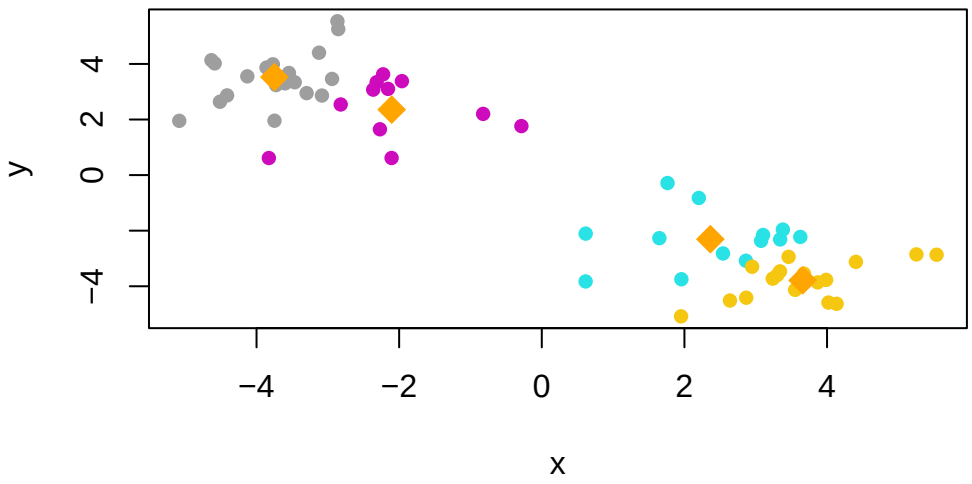
The "issue" is we tell `kmeans()` how many clusters we want!

> Q. Run k-means again and cluster into four groups/clusters and plot the results like we did above?

```
k4 <- kmeans(x, centers = 4)
k4
```

```
K-means clustering with 4 clusters of sizes 13, 11, 17, 19

Cluster means:
          x         y
1  2.363908 -2.306490
2 -2.104876  2.355539
3  3.657715 -3.788568
4 -3.749284  3.526371

Clustering vector:
 [1] 4 2 4 4 4 2 4 4 4 2 4 4 4 4 2 2 4 2 4 2 4 2 4 4 4 2 2 4 4 2 1 3 3 1 1 3 3 1
[39] 1 3 1 3 1 3 1 1 3 3 3 3 1 3 1 3 1 3 3 3 1 3

Within cluster sum of squares by cluster:
[1] 23.82872 20.28623 20.05693 23.72340
 (between_SS / total_SS =  93.3 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

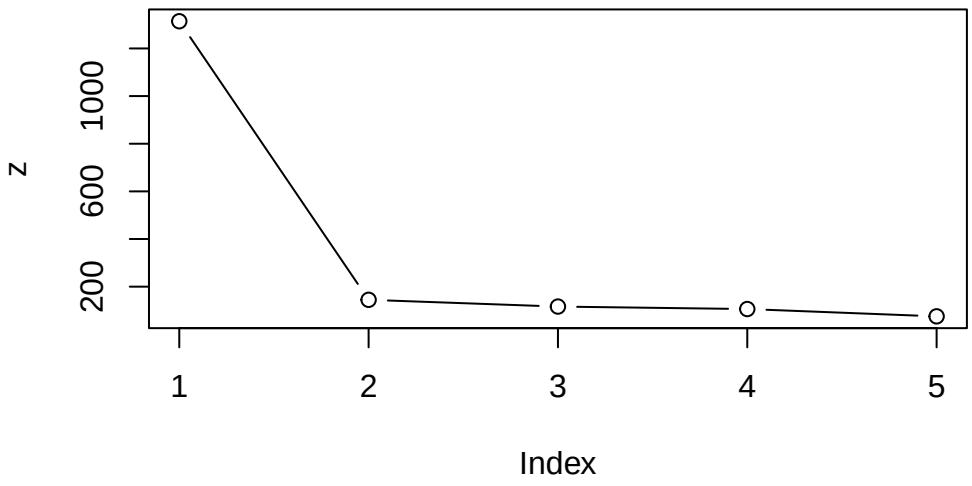
```
plot(x, col = 4 + k4$cluster, pch = 16)
points(k4$centers, col = "orange", pch = 18, cex = 2)
```



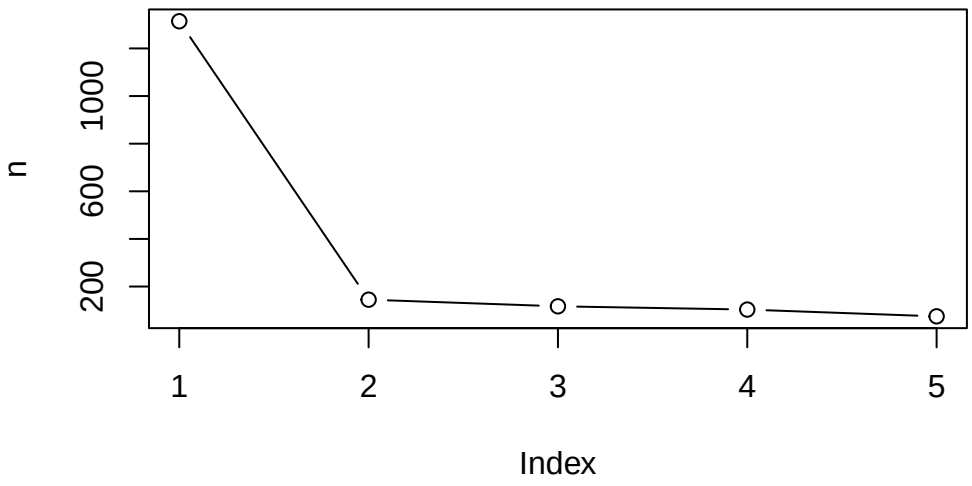
brute-force

```
k1 <- kmeans(x, center = 1)
k2 <- kmeans(x, center = 2)
k3 <- kmeans(x, center = 3)
k4 <- kmeans(x, center = 4)
k5 <- kmeans(x, center = 5)
```

```
z <- c(k1$tot.withinss, k2$tot.withinss,
k3$tot.withinss,
k4$tot.withinss,
k5$tot.withinss)

plot(z, typ = "b" )
```




```
n <- NULL
for (i in 1:5){
  n <- c(n, kmeans(x, centers = i)$tot.withinss)
}
plot(n, typ = "b")
```



Hierarchical Clustering

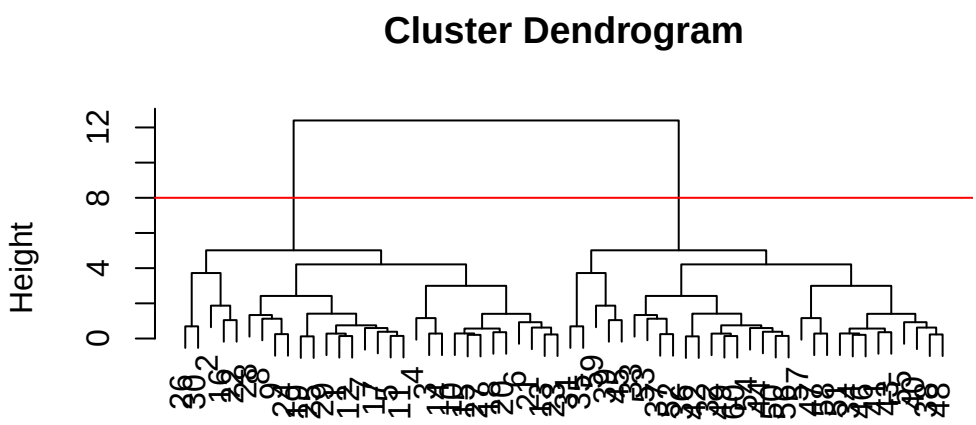
The main "base" R function for Hierarchical Clustering is called `hclust()`. Here we can't just input our data we need to first calculate a distance matrix (e.g. `dist()`) for our data and use this as input to `hclust()`.

```
d <- dist(x)
hc <- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a plot method for hclust results lets try it.

```
plot(hc)
abline(h = 8, col = "red")
```



To get our cluster "membership" vector (i.e. our main clustering result) we can "cut" the tree at a given heigh or at a height that yields a given "k" groups.

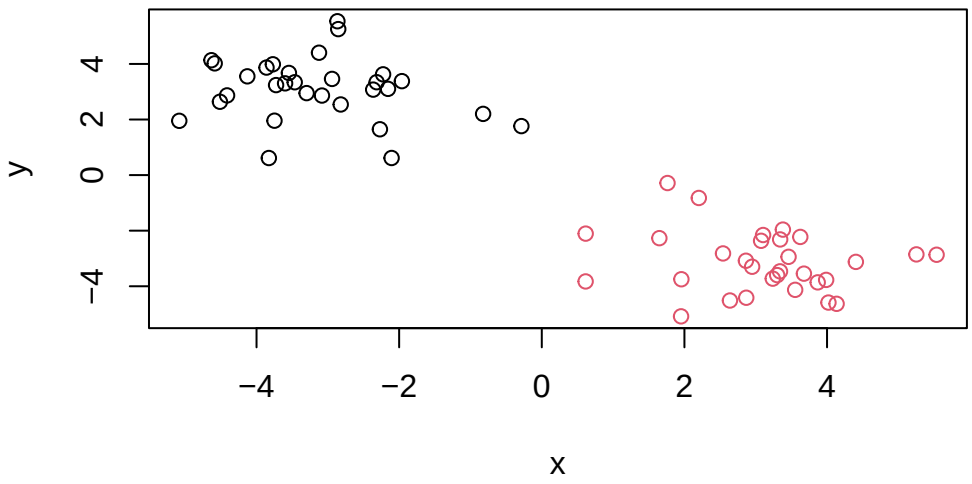
```
cutree(hc, h = 8)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
grps <- cutree(hc, k = 2)
```

Q. Plot the data with our hclust result coloring

```
plot(x, col=grps)
```



Principal Component Analysis (PCA)

PCA of UK food data

Import food data from an online CSV file:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
head(x)
```

```
              X England Wales Scotland N.Ireland
1        Cheese     105   103      103        66
2  Carcass_meat     245   227      242       267
3    Other_meat     685   803      750       586
4          Fish     147   160      122        93
5 Fats_and_oils     193   235      184       209
6        Sugars     156   175      147       139
```

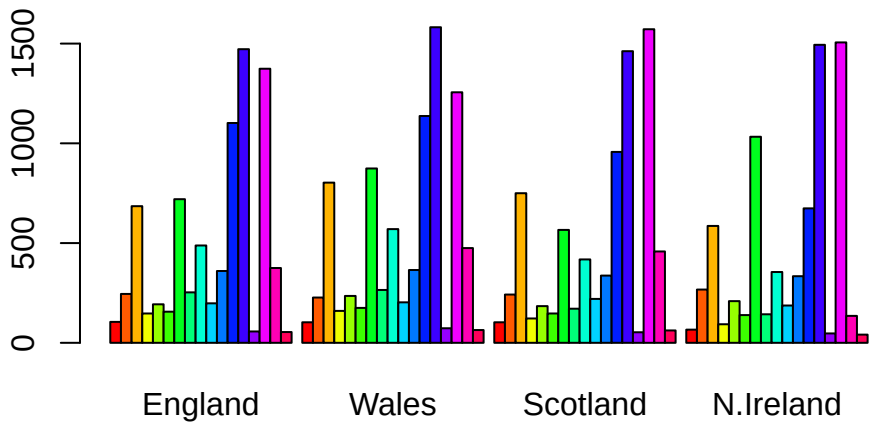
```
rownames(x) <- x [,1]
x <- x[,-1]
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

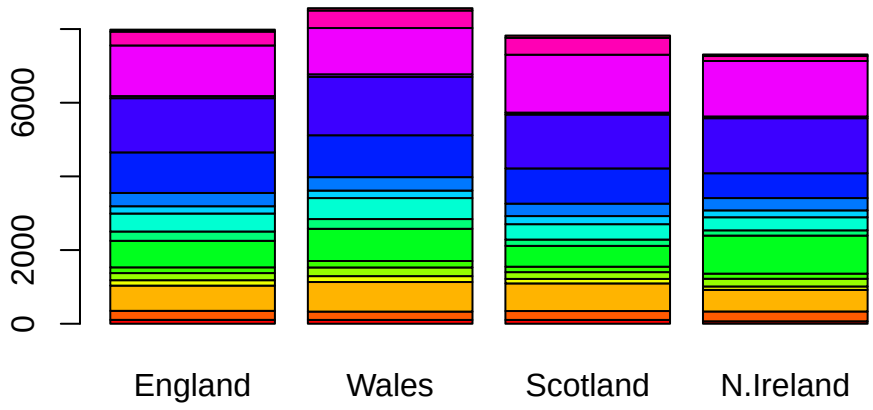
```
x <- read.csv(url, row.names = 1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

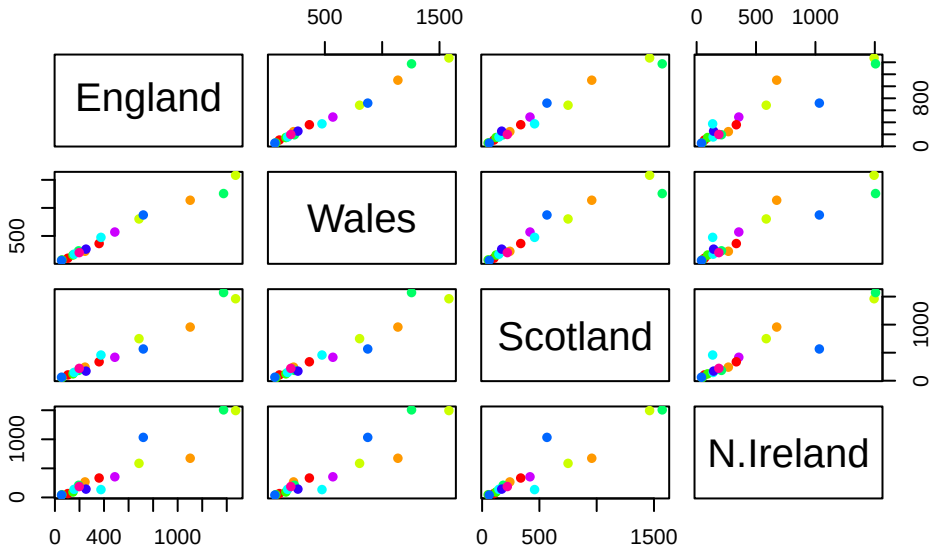
```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```




```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



```
pairs(x, col=rainbow(10), pch=16)
```



Main point: It can be difficult to spot major trends and patterns even in relatively small multivariate datasets (here we only have 17 dimensions, typically we have 1000s).

PCA to the rescue

The main function in "base" R for PCA is called `prcomp()`.

I will take the transpose of our food data so the "foods" are in the columns:

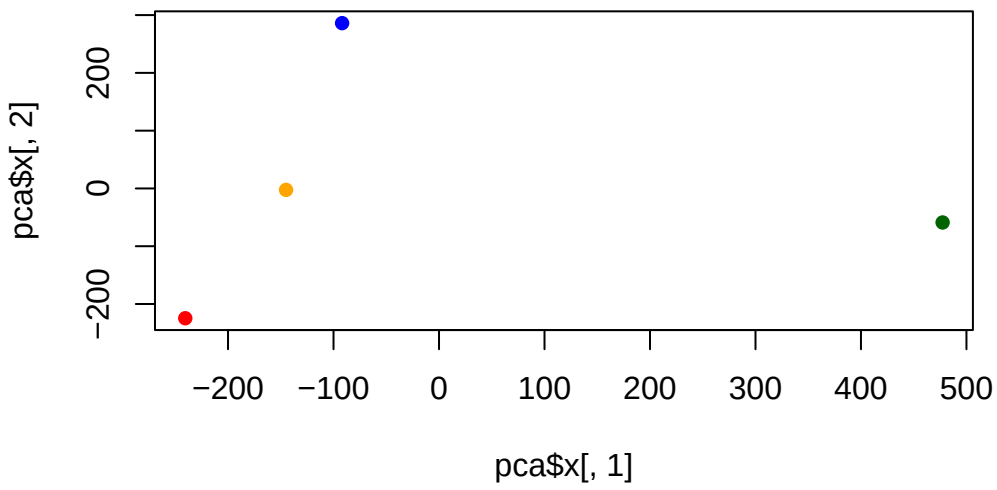
```
pca <- prcomp(t(x))
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 3.176e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

```
pca$x
```

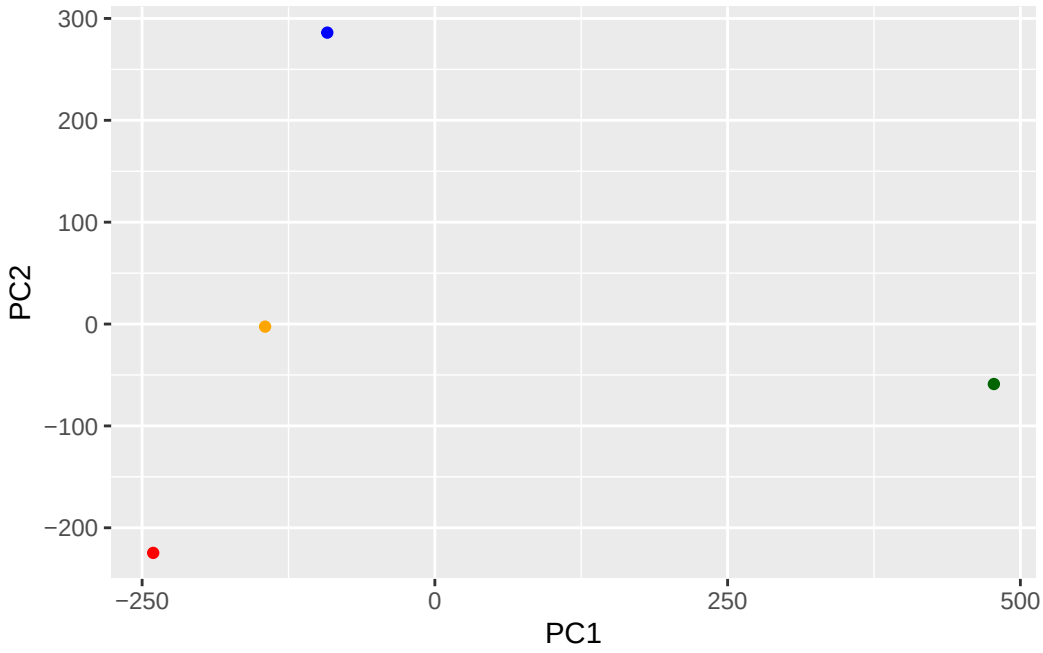
```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -4.894696e-14
Wales     -240.52915 -224.646925 -56.475555  5.700024e-13
Scotland   -91.86934  286.081786 -44.415495 -7.460785e-13
N.Ireland  477.39164  -58.901862  -4.877895  2.321303e-13
```

```
cols <- c("orange", "red", "blue", "darkgreen")
plot(pca$x[,1], pca$x[,2], col = cols, pch = 16)
```




```
library(ggplot2)
```

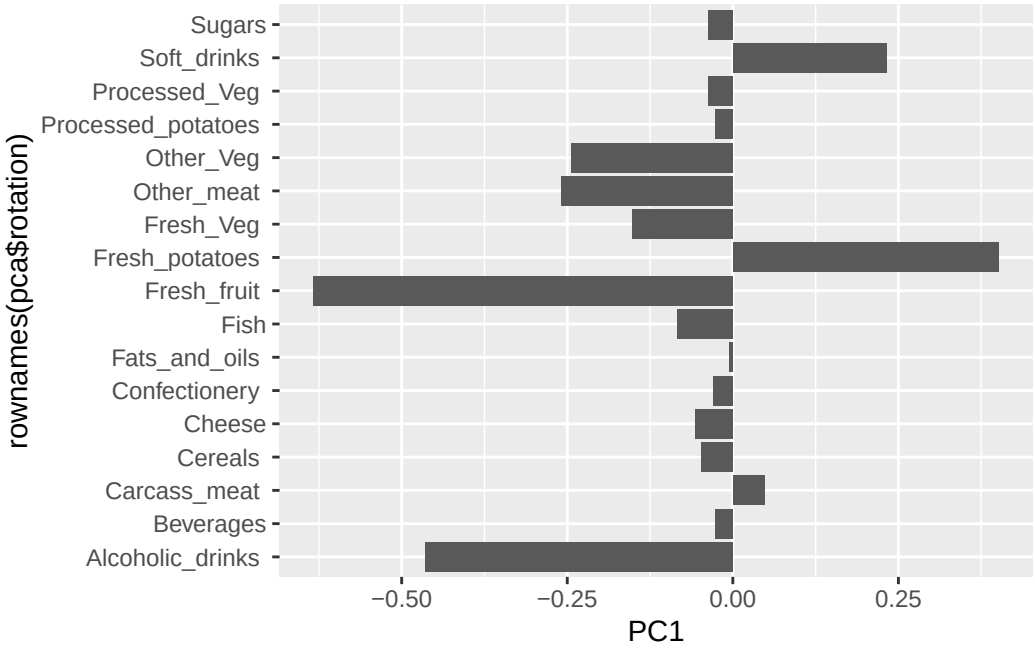
```
ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point(col=cols)
```




```
pca$rotation
```

	PC1	PC2	PC3	PC4
Cheese	-0.056955380	0.016012850	0.02394295	-0.694538519
Carcass_meat	0.047927628	0.013915823	0.06367111	0.489884628
Other_meat	-0.258916658	-0.015331138	-0.55384854	0.279023718
Fish	-0.084414983	-0.050754947	0.03906481	-0.008483145
Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.076097502
Sugars	-0.037620983	-0.043021699	-0.03605745	0.034101334
Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	-0.090972715
Fresh_Veg	-0.151849942	-0.144900268	0.21382237	-0.039901917
Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.016719075
Processed_potatoes	-0.026886233	0.042850761	-0.07364902	0.030125166
Processed_Veg	-0.036488269	-0.045451802	0.05289191	-0.013969507
Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.184072217
Cereals	-0.047702858	-0.212599678	-0.35884921	0.191926714
Beverages	-0.026187756	-0.030560542	-0.04135860	0.004831876
Soft_drinks	0.232244140	0.555124311	-0.16942648	0.103508492
Alcoholic_drinks	-0.463968168	0.113536523	-0.49858320	-0.316290619
Confectionery	-0.029650201	0.005949921	-0.05232164	0.001847469

```
ggplot(pca$rotation) +
  aes(PC1, rownames(pca$rotation)) +
  geom_col()
```



PCA looks super useful and we will come back to describe this further next day :-)